

Code Explanation

Test Suite Explanation: Customer Order SCD Type 2 ETL Job

Overview

This is a **comprehensive pytest test suite** for a PySpark-based ETL job that processes customer and order data with **SCD Type 2** (Slowly Changing Dimension) logic, writes to **Apache Hudi**, and registers tables in **AWS Glue Catalog**.---

Step-by-Step Breakdown

1. Imports and Setup

```
import pytest
from datetime import datetime, timedelta
from decimal import Decimal
from unittest.mock import Mock, patch, MagicMock
import yaml, tempfile, os
from pyspark.sql import SparkSession, DataFrame
from moto import mock_s3, mock_glue
import boto3
```

- **pytest**: Testing framework- **moto**: Mocks AWS services (S3, Glue) for local testing- **PySpark**: Data processing framework- **unittest.mock**: Mocks dependencies- Imports modules under test from `job.py` (SparkContextManager, S3DataManager, DataCleaner, etc.)---

2. Test Fixtures (Reusable Test Data)

2.1 Spark Session Fixture

```
@pytest.fixture(scope="session")
def spark_session():
    spark = SparkSession.builder.appName("TestCustomerOrderETL").master("local[2]").getOrCreate()
    yield spark
    spark.stop()
```

- Creates a **single Spark session** for all tests (session scope)- Configures local mode with 2 cores- Cleans up after all tests complete##### 2.2 Sample Data Fixtures- **sample_customer_data**: 5 customers with complete data- **sample_order_data**: 7 orders linked to customers- **sample_customer_with_nulls**: Data with NULL values for testing cleaning logic- **sample_customer_with_null_strings**: Data with "Null", "NULL", "None" strings- **sample_customer_with_duplicates**: Duplicate customer records with different timestamps##### 2.3 Configuration Fixtures

```
@pytest.fixture
def test_config():
    return { 'source_paths': {'customer': 's3://...', 'order': 's3://...'}, 'target_paths': {...}, 'hudi_config': {...}, 'scd_type2': {...}, 'data_cleaning': {...} }
```

- Provides test configuration matching production structure- **temp_config_file**: Writes config to temporary YAML file for testing file-based loading---

3. Spark Context Tests

Test: Spark Initialization

```
def test_spark_context_initialization():
    context_manager = SparkContextManager()
    sc, glue_ctx, spark = context_manager.initialize_spark_contexts()
    assert sc is not None
    assert glue_ctx is not None
```

- Verifies SparkContext, GlueContext, and SparkSession are properly initialized- Tests **FR-INIT-001** requirement---

****4. S3 Data Operations Tests****

****Test: S3 Path Validation****

```
@mock_s3def test_s3_path_validation(): s3_client = boto3.client('s3',
region_name='us-east-1') s3_client.create_bucket(Bucket='adif-sdlc') s3_manager =
S3DataManager(spark) assert
s3_manager.validate_s3_path("s3://adif-sdlc/curated/...") == True
```

- Uses ****moto**** to mock S3 service- Tests valid/invalid S3 path detection- Tests ****FR-S3-001**** requirement##### ****Test: Safe Data Reading****

```
def test_read_data_safe_with_schema(spark_session, sample_customer_data,
tmp_path): temp_path = str(tmp_path / "customer_data")
sample_customer_data.write.parquet(temp_path) df =
s3_manager.read_data_safe(temp_path, file_format="parquet") assert df.count() == 5
```

- Writes test data to temporary location- Reads back with schema validation- Tests ****FR-S3-002**** requirement---

****5. Data Cleaning Tests****

****Test: Remove NULL Values****

```
def test_remove_nulls(sample_customer_with_nulls): cleaner = DataCleaner()
df_cleaned = cleaner.remove_nulls(sample_customer_with_nulls, ['customer_id',
'customer_name', 'email']) assert df_cleaned.count() == 2 # Only rows with all
required fields
```

- Filters out rows with NULL in required fields- Tests ****FR-CLEAN-001**** requirement##### ****Test: Remove "Null" Strings****

```
def test_remove_null_strings(sample_customer_with_null_strings): df_cleaned =
cleaner.remove_null_strings(df, ['Null', 'NULL', 'None']) null_email_count =
df_cleaned.filter(col("email").isNull()).count() assert null_email_count >= 1
```

- Replaces string literals like "Null", "NULL" with actual NULL values- Tests ****FR-CLEAN-002**** requirement##### ****Test: Remove Duplicates****

```
def test_remove_duplicates(sample_customer_with_duplicates): df_deduped =
cleaner.remove_duplicates(df, key_columns=['customer_id'],
order_column='registration_date') assert df_deduped.count() == 3 # 3 unique
customers
```

- Keeps only the ****latest record**** per customer_id based on timestamp- Tests ****FR-CLEAN-003**** requirement---

****6. SCD Type 2 Tests****

****Test: Add SCD2 Columns****

```
def test_add_scd2_columns(sample_customer_data): processor = SCD2Processor()
df_scd2 = processor.add_scd2_columns(sample_customer_data) assert "IsActive" in
df_scd2.columns assert "StartDate" in df_scd2.columns assert "EndDate" in
df_scd2.columns assert "OpTs" in df_scd2.columns
```

- Adds 4 SCD Type 2 tracking columns: - ****IsActive****: Boolean flag (current record = True) - ****StartDate****: Record effective start timestamp - ****EndDate****: Record expiration timestamp (NULL for active) - ****OpTs****: Operation timestamp for versioning- Tests ****FR-SCD2-001**** requirement##### ****Test: Validate Column Types****

```
def test_scd2_column_types(sample_customer_data): schema_dict = {field.name:
field.dataType for field in df_scd2.schema.fields} assert
```

```
isinstance(schema_dict['IsActive'], BooleanType) assert
isinstance(schema_dict['StartDate'], TimestampType)
```

- Verifies correct data types for SCD2 columns- Tests ****FR-SCD2-003**** requirement---

****7. Customer Aggregation Tests****

****Test: Calculate Aggregate Spend****

```
def test_customer_aggregate_spend_calculation(sample_customer_data,
sample_order_data): aggregator = CustomerAggregator() agg_df =
aggregator.calculate_customer_aggregate_spend(customer_df, order_df) c001_metrics
= agg_df.filter(col("customer_id") == "C001").collect()[0] assert
c001_metrics['total_orders'] == 3 assert float(c001_metrics['total_spend']) ==
475.0
```

- Joins customer and order data- Calculates per-customer metrics: - ****total_orders****: Count of orders - ****total_spend****: Sum of order amounts - ****avg_order_value****: Average order amount - ****first_order_date****: Earliest order timestamp - ****last_order_date****: Latest order timestamp- Tests ****FR-AGG-001**** requirement---

****8. Hudi Configuration Tests****

```
def test_hudi_write_configuration(test_config): hudi_config =
test_config['hudi_config'] assert hudi_config['table_type'] == 'COPY_ON_WRITE'
assert hudi_config['operation'] == 'upsert' assert hudi_config['precombine_field']
== 'OpTs'
```

- Validates Hudi table configuration: - ****COPY_ON_WRITE****: Table type for read-optimized performance - ****upsert****: Merge operation (insert + update) - ****OpTs****: Field used to determine latest record version- Tests ****FR-HUDI-001**** requirement---

****9. Glue Catalog Tests****

****Test: Create Database****

```
@mock_gluedef test_create_glue_database(): catalog_manager =
GlueCatalogManager('sdlc_wizard_db') success =
catalog_manager.create_database if not exists() assert success == True
```

- Uses ****moto**** to mock AWS Glue- Creates Glue Catalog database- Tests ****FR-CATALOG-001**** requirement#### ****Test: Register Table****

```
@mock_gluedef test_register_table_in_catalog(): columns = [{'Name': 'customer_id',
'Type': 'string'}, ...] success =
catalog_manager.register_table(table_name='customer_aggregate_spend',
s3_path='s3://...', columns=columns)
```

- Registers table metadata in Glue Catalog- Tests ****FR-CATALOG-002**** requirement---

****10. Error Handling Tests****

```
def test_handle_missing_s3_path(spark_session): df =
s3_manager.read_data_safe('s3://nonexistent/path/') assert df is None
def test_handle_empty_dataframe(spark_session): empty_df =
spark_session.createDataFrame([], schema) result = cleaner.remove_nulls(empty_df,
['customer_id']) assert result.count() == 0
```

- Tests graceful handling of: - Missing S3 paths - Invalid schemas - Empty DataFrames- Tests
FR-ERROR-001/002/003 requirements---

****11. End-to-End Workflow Tests****

```
def test_complete_data_pipeline(sample_customer_data, sample_order_data): # Stage 1: Clean customer_cleaned = cleaner.remove_nulls(customer_df, [...]) # Stage 2: Add SCD2 customer_scd2 = processor.add_scd2_columns(customer_cleaned) # Stage 3: Aggregate agg_df = aggregator.calculate_customer_aggregate_spend(customer_scd2, order_scd2) # Verify assert processor.validate_scd2_columns(agg_scd2)
```

- Tests complete pipeline: **Clean → SCD2 → Aggregate**- Validates data flows correctly through all stages- Tests **FR-E2E-002** requirement---

****12. Summary Validation Test****

```
def test_all_trd_requirements_implemented(test_config): # Verify S3 paths assert test_config['source_paths']['customer'] == 's3://adif-sdlc/curated/...' # Verify SCD2 config assert 'is_active' in scd2_columns # Verify Hudi config assert test_config['hudi_config']['operation'] == 'upsert'
```

- **Final validation** that all TRD (Technical Requirements Document) requirements are implemented- Prints summary report of validated components---

Key Testing Patterns Used

1. ****Fixtures****: Reusable test data and configurations2. ****Mocking****: AWS services (S3, Glue) mocked with `moto`3. ****Parametrization****: Multiple test cases with different data scenarios4. ****Isolation****: Each test is independent and can run in any order5. ****Assertions****: Explicit validation of expected outcomes6. ****Traceability****: Each test maps to a functional requirement (FR-XXX-XXX)---

Test Coverage Areas

■ Spark initialization ■ S3 read/write operations ■ Data cleaning (nulls, duplicates, invalid strings) ■ SCD Type 2 column management ■ Customer aggregation logic ■ Hudi configuration ■ Glue Catalog integration ■ Error handling ■ End-to-end pipeline execution ■ Schema validation ■ Performance optimization (caching, partitioning)